

Dev and Cloud architecture

Model: qwen/qwen3.6-35b-a3b Created: 6/11/2026, 2:00:48 PM

User

Give me a high level architecture of the multi-tenant portal on a developers machine and a cloud platform eg AWS

Assistant

Here's a high-level architecture for a multi-tenant portal, split into **local developer machine** and **AWS cloud** deployments. The baseline assumption is a **shared database with row-level isolation** (most common for cost-effective SaaS), but I'll note alternatives where relevant.

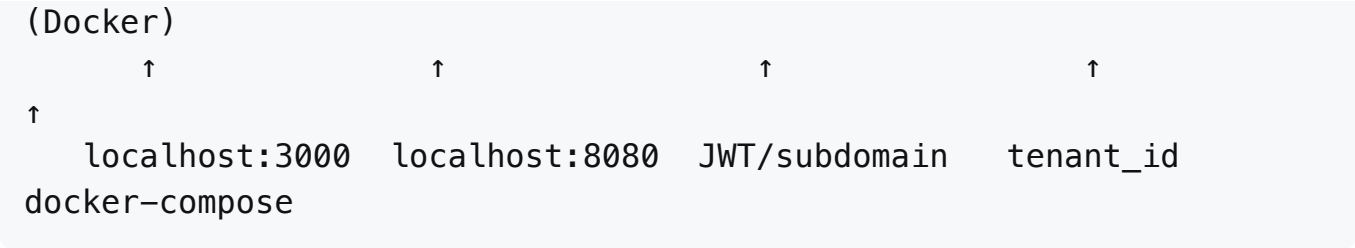
🔑 Core Multi-Tenancy Concepts (Across Both Environments)

Concept	How It Works
Tenant Identification	Extracted from JWT <code>tenant_id</code> , subdomain, or request header
Context Propagation	Injected into DB session, service calls, and logging
Data Isolation	Row-Level Security (RLS), schema-per-tenant, or DB-per-tenant
Routing	API Gateway/ALB routes to tenant-aware middleware → services
Config/Secrets	Environment variables (local) → AWS Parameter Store/Secrets Manager (cloud)

💻 1. Developer Machine Architecture (Local)

Lightweight, reproducible, and simulates cloud patterns without AWS costs.





Key Components & Tools

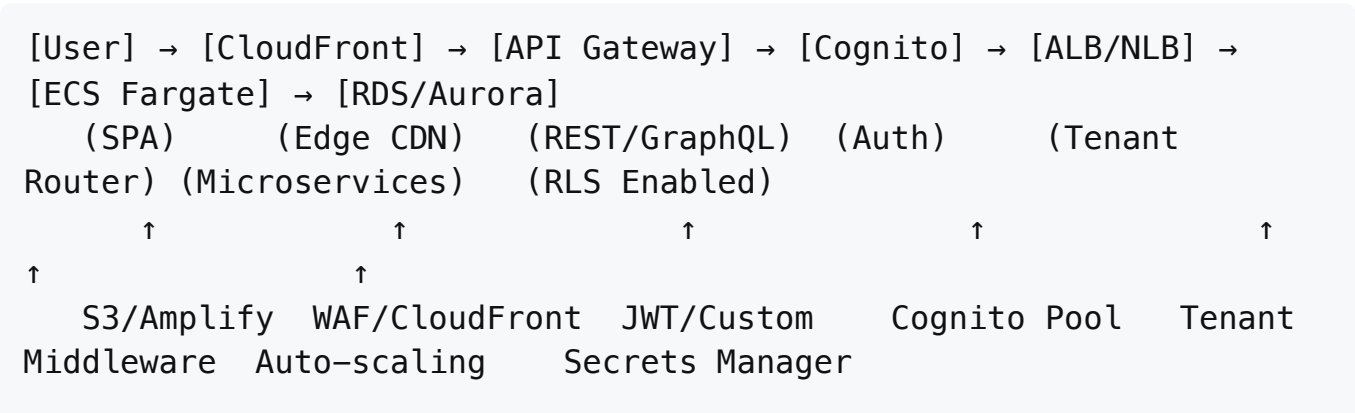
Component	Local Implementation
Frontend	React/Vue/Angular on localhost:3000
Auth	Keycloak Docker or local JWT issuer
API Gateway	Express/NestJS/FastAPI with tenant middleware
Tenant Routing	Extracts tenant_id from JWT/subdomain → sets context
Database	PostgreSQL via Docker. Uses tenant_id column + RLS or separate schemas
Config	.env files, dotenv
Orchestration	docker-compose.yml (DB, API, Auth, optional LocalStack)

Tenant Context Flow (Local)

- 1. User logs in → receives JWT with tenant_id
- 2. API middleware validates JWT → extracts tenant_id
- 3. tenant_id injected into DB session (via RLS policy or query filter)
- 4. All service calls pass tenant_id in context (e.g., ctx.tenantId)
- 5. LocalStack (optional) simulates S3, SQS, Secrets Manager if needed

☁️ 2. AWS Cloud Architecture (Production SaaS)

Scalable, secure, and managed. Uses standard AWS SaaS patterns.



Key AWS Components & Mapping

Component	AWS Service	Multi-Tenancy Role
Frontend	S3 + CloudFront + Amplify	Static hosting, edge caching, custom domains per tenant
Auth	Cognito User Pools + Identity Pools	Issues JWT with tenant_id, role, sub
API Gateway	API Gateway + Lambda@Edge	Routes, throttles per tenant, custom authorizer
Tenant Router	Custom Authorizer → API Gateway → ALB → ECS	Extracts tenant_id, validates, injects into request context
Compute	ECS Fargate (or Lambda)	Tenant-aware service layer. Each request carries tenant_id
Database	RDS PostgreSQL or Aurora + RLS	Row-Level Security enforces WHERE tenant_id = current_setting('app.tenant_id')
Cache/Queue	ElastiCache (Redis) + SQS/SNS	Tenant-scoped keys, message routing
Config/Secrets	Parameter Store + Secrets Manager	Per-tenant or shared config, encrypted at rest
Observability	CloudWatch + X-Ray + OpenTelemetry	Tenant-tagged logs, traces, metrics
Security	WAF, Shield, IAM, KMS, VPC	Network isolation, DDoS protection, least-privilege access

Tenant Context Flow (AWS)

1. User authenticates via Cognito → gets JWT with tenant_id
 2. API Gateway Custom Authorizer validates JWT → extracts tenant_id
 3. Request forwarded to ALB → ECS task with tenant middleware
 4. Middleware sets DB session variable: SET app.tenant_id = 't_123'
 5. PostgreSQL RLS automatically filters queries: WHERE tenant_id = current_setting('app.tenant_id')
 6. All downstream services inherit context via headers, traces, or distributed context propagation
-

Multi-Tenancy Data Isolation Patterns (Choose Based on Requirements)

Pattern	Pros	Cons	Best For
Shared DB + RLS	Cost-efficient, easy backups, cross-tenant analytics	DB performance isolation, complex RLS policies	Most SaaS apps
Schema-per-tenant	Better isolation, easier backups/migrations per tenant	Schema management overhead	Mid-size SaaS, compliance-heavy
DB-per-tenant	Max isolation, independent scaling/backups	High cost, complex provisioning	Enterprise, regulated industries

Key Implementation Considerations

| Area | Recommendation | |-----| | **Tenant Onboarding** | Provision DB schema/RLS policies via IaC (Terraform), or use database migrations with tenant ID seeding | | **Connection Pooling** | Use PgBouncer or AWS RDS Proxy to avoid tenant connection exhaustion | | **Multi-Domain Routing** | Use CloudFront + Lambda@Edge or API Gateway custom domains to route `tenant1.app.com` → correct context | | **Cross-Tenant Queries** | Avoid unless absolutely necessary. If needed, use admin role bypass or dedicated analytics DB | | **Scaling** | ECS Fargate auto-scales by CPU/memory. RLS adds ~5-10% query overhead; index on `tenant_id` is mandatory | | **Security** | RLS + IAM + WAF + KMS. Never trust client-side tenant ID. Validate at API Gateway & DB layer |

Suggested Local Dev Stack vs AWS Stack

Layer	Local	AWS
Frontend	Vite/React → <code>localhost:3000</code>	S3 + CloudFront + Amplify
Auth	Keycloak Docker / local JWT	Cognito + Lambda@Edge authorizer
API	Express/NestJS + tenant middleware	API Gateway + ALB + ECS Fargate
DB	PostgreSQL (Docker) + RLS	Aurora Serverless v2 + RLS
Cache/Queue	Redis Docker + local SQS	ElastiCache + SQS/SNS
Config	<code>.env</code>	Parameter Store + Secrets Manager
CI/CD	GitHub Actions → Docker	CodePipeline / GitHub Actions → ECR + ECS/Aurora

✅ Next Steps to Implement

1. **Pick isolation strategy** (start with Shared DB + RLS)
2. **Define tenant context contract** (JWT claims, header names, DB session vars)
3. **Build tenant router middleware** (extracts tenant_id, validates, injects)
4. **Enable RLS in PostgreSQL** (CREATE POLICY tenant_isolation ON ... USING (tenant_id = current_setting('app.tenant_id')))
5. **Containerize & compose** locally → test multi-tenant flows
6. **Deploy to AWS** using Terraform/CDK → automate tenant provisioning pipeline

Would you like a Terraform/CDK snippet for the AWS multi-tenant RDS + RLS setup, or a Docker Compose template for local dev?

User

yes

Assistant

Here's a production-aligned, copy-paste-ready implementation for both environments. I've structured it to enforce multi-tenancy at the database layer while keeping local dev lightweight.

🌐 Part 1: Terraform (AWS Multi-Tenant RDS + RLS + RDS Proxy)

Focus: Secure, scalable infra with RLS enforcement and connection pooling.

providers.tf

```
terraform {
  required_providers {
    aws = { source = "hashicorp/aws", version = "> 5.0" }
  }
}

provider "aws" {
  region = var.region
}
```

variables.tf

```
variable "region"      { default = "us-east-1" }
variable "vpc_cidr"    { default = "10.0.0.0/16" }
variable "db_username" { type      = string; sensitive = true }
variable "db_password" { type      = string; sensitive = true }
variable "tenant_id"   { type      = string; default   = "a1b2c3d4-
e5f6-7890-abcd-ef1234567890" }
```

vpc.tf

```
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true
}

resource "aws_subnet" "a" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.1.0/24"
  availability_zone = "${var.region}a"
}

resource "aws_subnet" "b" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.2.0/24"
  availability_zone = "${var.region}b"
}

resource "aws_security_group" "rds" {
  name     = "saas-rds-sg"
  vpc_id   = aws_vpc.main.id

  ingress {
    from_port = 5432
    to_port   = 5432
    protocol  = "tcp"
    cidr_blocks = ["10.0.0.0/16"]
  }
}
```

rds.tf

```
resource "aws_db_instance" "multi_tenant" {
  identifier          = "saas-multi-tenant-db"
  engine              = "postgres"
  engine_version      = "15"
  instance_class      = "db.t3.medium"
  db_name             = "saasdb"
  username            = var.db_username
  password            = var.db_password
  skip_final_snapshot = true
  publicly_accessible = false
  storage_encrypted   = true
  db_subnet_group_name = aws_db_subnet_group.main.name
  vpc_security_group_ids = [aws_security_group.rds.id]
}

resource "aws_db_subnet_group" "main" {
  subnet_ids = [aws_subnet.a.id, aws_subnet.b.id]
}
```

rds_proxy.tf

```
resource "aws_rds_proxy" "multi_tenant" {
  name          = "saas-rds-proxy"
  role_arn      = aws_iam_role.rds_proxy.arn
  vpc_subnet_ids = [aws_subnet.a.id, aws_subnet.b.id]
  vpc_security_group_ids = [aws_security_group.rds.id]
  target_db_instance {
    db_instance_identifier =
aws_db_instance.multi_tenant.identifier
  }
}

resource "aws_iam_role" "rds_proxy" {
  name = "rds-proxy-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = { Service = "rds.amazonaws.com" }
      Action = "sts:AssumeRole"
    }]
  })
}
```

```

    }]
  })
}

resource "aws_iam_role_policy_attachment" "rds_proxy" {
  role      = aws_iam_role.rds_proxy.name
  policy_arn = "arn:aws:iam::aws:policy/service-
role/AmazonRDSDatabaseProxyFullAccess"
}

```

params.tf (Tenant Context Config)

```

resource "aws_ssm_parameter" "tenant_context" {
  name  = "/saas/app/tenant_id"
  type  = "String"
  value = var.tenant_id
}

```

Production Notes (Terraform)

- **RLS is applied via SQL migrations** (Flyway/Liquibase), not Terraform. Terraform manages infra; migrations manage schema/policies.
- **ECS Task IAM Role** must have `ssm:GetParameter` to read tenant context.
- **CDK Equivalent:** Use `aws_rds.CfnDBInstance`, `aws_rds.CfnDBProxy`, `aws_rds.CfnDBProxyTargetGroup`, and `aws_rds.CfnDBProxyTarget` with `@aws-cdk/aws-rds` module.

Part 2: Docker Compose (Local Dev with RLS)

Focus: Zero-cost multi-tenant simulation with automatic RLS setup.

docker-compose.yml

```

version: '3.8'
services:
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: saas_dev

```

```
POSTGRES_PASSWORD: devpass123
POSTGRES_DB: saasdb
ports:
  - "5432:5432"
volumes:
  - ./init-rls.sql:/docker-entrypoint-initdb.d/init-rls.sql
  - pgdata:/var/lib/postgresql/data
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U saas_dev"]
  interval: 10s
  timeout: 5s
  retries: 5
networks:
  - saas-net
restart: unless-stopped
```

```
volumes:
  pgdata:

networks:
  saas-net:
    driver: bridge
```

init-rls.sql

```
CREATE EXTENSION IF NOT EXISTS "uuid-osspl";

-- Tenant table
CREATE TABLE tenants (
  id uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  name text NOT NULL,
  created_at timestamptz DEFAULT now()
);

-- Shared resource table
CREATE TABLE resources (
  id uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id uuid REFERENCES tenants(id),
  data jsonb,
  created_at timestamptz DEFAULT now()
);
```

```
-- 🗝️ Enable Row-Level Security
ALTER TABLE resources ENABLE ROW LEVEL SECURITY;

-- 🛡️ RLS Policy: Enforces tenant isolation
CREATE POLICY tenant_isolation_policy ON resources
  FOR ALL
  USING (tenant_id = current_setting('app.tenant_id', true)::uuid)
  WITH CHECK (tenant_id = current_setting('app.tenant_id',
true)::uuid);

-- 🔑 Grant permissions
GRANT USAGE ON SCHEMA public TO saas_dev;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public
TO saas_dev;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO saas_dev;

-- 🌱 Seed data
INSERT INTO tenants (id, name) VALUES
  ('a1b2c3d4-e5f6-7890-abcd-ef1234567890', 'Acme Corp'),
  ('b2c3d4e5-f6a7-8901-bcde-f12345678901', 'Globex Inc');

INSERT INTO resources (tenant_id, data) VALUES
  ('a1b2c3d4-e5f6-7890-abcd-ef1234567890', '{"project": "Alpha"}'),
  ('b2c3d4e5-f6a7-8901-bcde-f12345678901', '{"project": "Beta"}');
```

🔧 How to Test Multi-Tenant Isolation Locally

```
# Start stack
docker compose up -d

# Tenant A context
psql -h localhost -U saas_dev -d saasdb -c "SET app.tenant_id =
'a1b2c3d4-e5f6-7890-abcd-ef1234567890'; SELECT * FROM resources;"

# Tenant B context
psql -h localhost -U saas_dev -d saasdb -c "SET app.tenant_id =
'b2c3d4e5-f6a7-8901-bcde-f12345678901'; SELECT * FROM resources;"
```

✅ You'll see only the seeded row for each tenant. Cross-tenant access returns 0 rows.



How Apps Should Inject Tenant Context

Multi-tenancy fails if context isn't propagated. Here's the contract:

Layer	Implementation
API Gateway / Middleware	Extracts <code>tenant_id</code> from JWT → validates → sets DB session
Connection Pool (Node/Python)	SET <code>app.tenant_id = '...'</code> on checkout, or pool config options: <code>'-c app.tenant_id=...'</code>
ORM (Prisma/TypeORM/SQLAlchemy)	Use middleware/hooks to inject session variable before every query
PostgreSQL RLS	<code>current_setting('app.tenant_id', true)</code> safely reads the session var

Example (Node.js + pg):

```
const client = await pool.connect();
await client.query("SET app.tenant_id = $1", [tenantId]);
const res = await client.query("SELECT * FROM resources"); //
// Automatically filtered by RLS
client.release();
```

Example (Python + SQLAlchemy):

```
engine = create_engine(conn_str, connect_args={"options": "-c
app.tenant_id=abc123"})
```



Critical Production Checklist

Area	Recommendation
RLS Enforcement	Never rely solely on app-layer filtering. RLS is the final guardrail.
Tenant Context Validation	Validate <code>tenant_id</code> at API Gateway → Cognito → ECS middleware → DB session. Reject if missing/mismatched.
Connection Pooling	Use RDS Proxy + PgBouncer. Set <code>max_conn</code> per tenant to prevent exhaustion.
Cross-Tenant Queries	Block by default. Use admin role with <code>SET app.tenant_id = NULL;</code> or dedicated analytics DB.

Area	Recommendation
Secrets	Store DB creds in AWS Secrets Manager. Rotate quarterly. Never in .env or code.
Monitoring	Tag CloudWatch logs/X-Ray traces with tenant_id. Alert on RLS bypass attempts.

Next Steps

1. Run `docker compose up` → verify RLS isolation with `psql`
2. Apply RLS via Flyway/Liquibase in production (never manual `init-rls.sql`)
3. Configure ECS task to read tenant context from SSM/Env
4. Add WAF rules to throttle per-tenant API abuse
5. Implement tenant onboarding pipeline (provision SSM params, RLS policies, metrics)

Want the **Flyway migration structure**, **ECS task definition with RDS Proxy**, or **Cognito JWT tenant claim setup** next?